Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №8

по курсу «Логика и основы алгоритмизации в инженерных задачах»

на тему «Обход графа в ширину»

Выполнили:
студенты группы 25ВВВ3
Шибанов Ф.Д.
Флеонов М.В.

Пенза 2026

Цель работы и описание алгоритмов

Цель работы: изучить обход графа в ширину, реализовать его для матрицы и списков смежности, самостоятельно построить очередь на связном списке и сравнить время двух реализаций очереди для графов разных порядков. Выполнены основные и все дополнительные задания.

Представление графа

Вершины обозначаются числами от 1 до n. Матрица смежности содержит нули при отсутствии ребра. Для неориентированного графа она симметрична, диагональ нулевая. Внутри программы используются индексы от 0 до n − 1; при выводе номера увеличиваются на единицу.

Списки смежности построены как односвязные списки: узел хранит номер соседней вершины и ссылку на следующий узел. Соседи просматриваются по возрастанию номеров. Преобразование матрицы в списки выполняется один раз до запуска алгоритмов.

Генерация графа

Матрица генерируется функцией rand. В начале программы выполняется srand((unsigned)time(NULL)), чтобы начальное состояние генератора зависело от текущего времени. Для каждой пары разных вершин вероятность наличия ребра задаётся параметром p в процентах. В неориентированном графе значения записываются симметрично. Пользователю не требуется задавать начальное состояние генератора.

Принцип обхода в ширину

Начальная вершина помечается и помещается в FIFO-очередь. Затем вершина извлекается из начала очереди, а её ещё не посещённые соседи помечаются и добавляются в конец. Пометка выполняется при добавлении, поэтому вершина не попадает в очередь повторно. После завершения компоненты поиск возобновляется из первой непосещённой вершины, если нужен полный обход графа.

Среда выполнения

Программа написана на C++. Для сборки достаточно команды g++ main.cpp -o lab8. Запуск и измерения выполнены в Linux x86_64. Порядок графа ограничен 2000 вершинами для ограничения объёма матрицы.

Задание 1 Генерация и обход графа

Пункты 1 и 2 Матрица и стандартная очередь

Сгенерирована симметричная матрица неориентированного невзвешенного графа. Число вершин n (1..2000) и вероятность ребра p (0..100%) вводятся с клавиатуры. Обход всегда начинается с вершины 1. Используется std::queue<int>: push добавляет вершину, front возвращает первую, pop удаляет её.

Пункт 3 со звёздочкой Списки смежности

Вместо просмотра всей строки матрицы просматриваются узлы списка текущей вершины. Правила пометки и работы очереди сохраняются. Одна функция traverse реализует четыре режима: матрица и std::queue, матрица и своя очередь, списки и std::queue, списки и своя очередь. Программа выводит матрицу, списки и четыре порядка обхода.



Рисунок 1 — Матрица, списки и результаты четырёх вариантов BFS

При n = 6, p = 40% и старте из вершины 1 получен одинаковый порядок 1 2 3 5 4 6. Все четыре варианта посещают каждую вершину ровно один раз. При повторном запуске случайный граф и порядок обхода могут измениться.

Задание 2 Собственная очередь

Пункт 1 со звёздочкой Очередь на связном списке

Узел Node содержит номер вершины vertex и указатель next. Очередь LinkedQueue хранит указатели head и tail, указатель pool на заранее выделенный массив из n узлов и счётчик used. При push берётся следующий узел массива и присоединяется к tail; в пустой очереди он становится также head. При pop указатель head сдвигается, а при опустошении очереди tail обнуляется.

Узлы при извлечении не освобождаются по отдельности. Каждая вершина добавляется не более одного раза, поэтому на полный обход достаточно n узлов. При новом вызове traverse счётчик used начинается с нуля; весь массив освобождается в main после измерений. Собственная очередь используется для обоих представлений графа.

Проверка несвязного графа

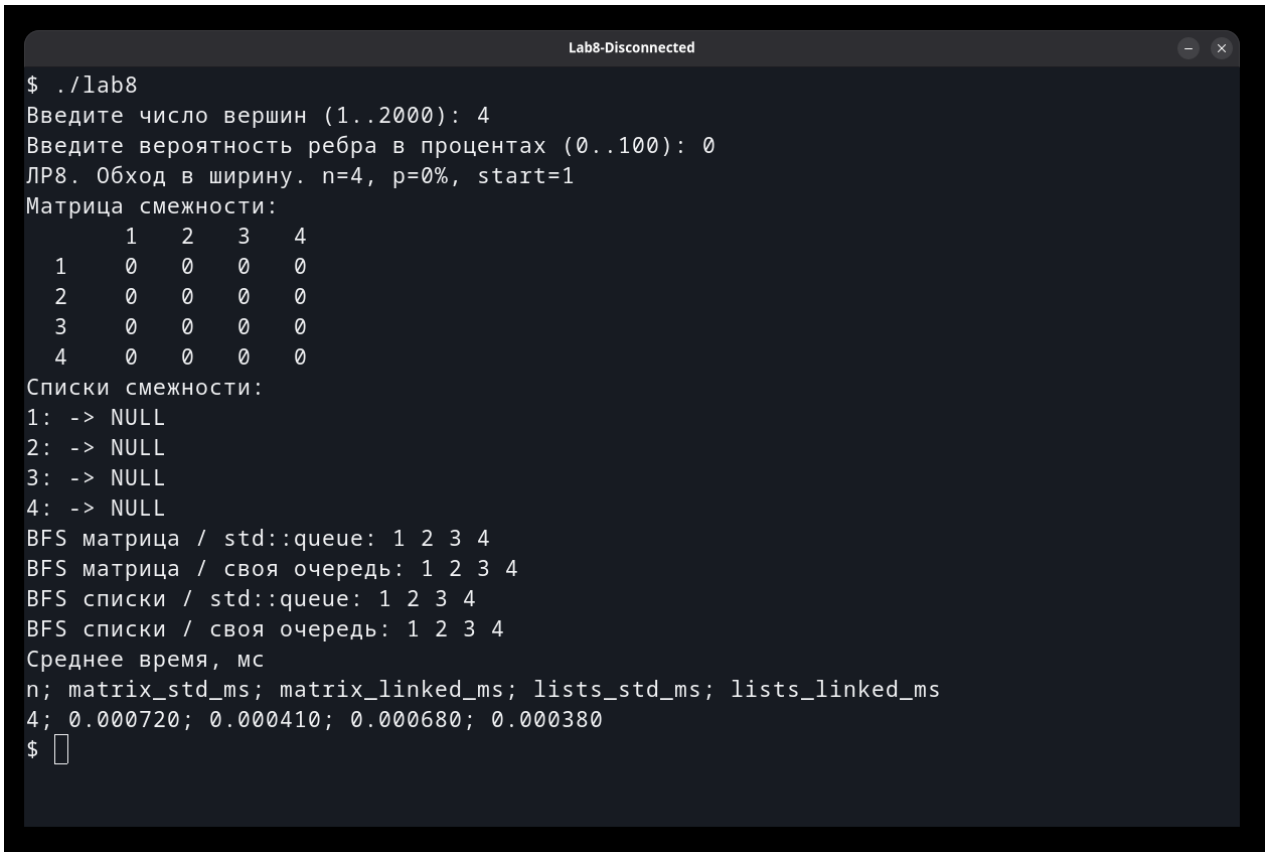


Рисунок 2 — Обход четырёх изолированных вершин из вершины 1

При n = 4 и p = 0 рёбра отсутствуют. После вершины 1 внешний цикл выбирает непосещённые вершины 2, 3 и 4. Все четыре реализации выводят 1 2 3 4. Граф содержит четыре компоненты связности, однако программа отдельно не вычисляет и не выводит их количество.

Оценка времени работы

Замеры: среднее время 100 полных обходов; p = 20%; время CPU в миллисекундах измеряется функцией clock. Сборка: g++ main.cpp -o lab8, без флагов оптимизации.

Для каждого порядка запускается программа и генерируется один граф, общий для четырёх режимов. Генерация, построение списков, выделение массивов order, seen и pool и вывод исключены из замера. Включены сброс seen, создание локальных объектов очередей, операции очереди и полный обход. Каждый режим запускается 100 раз, затем суммарное время делится на 100. Значения зависят от графа и условий выполнения.

n	Матрица и std::queue, мс	Матрица и своя очередь, мс	Списки и std::queue, мс	Списки и своя очередь, мс
100	0.080850	0.071370	0.021720	0.020270
300	0.743740	0.599300	0.076560	0.087600
600	2.683930	3.060060	0.536820	0.500050
1000	7.463930	7.360310	1.311380	1.349470

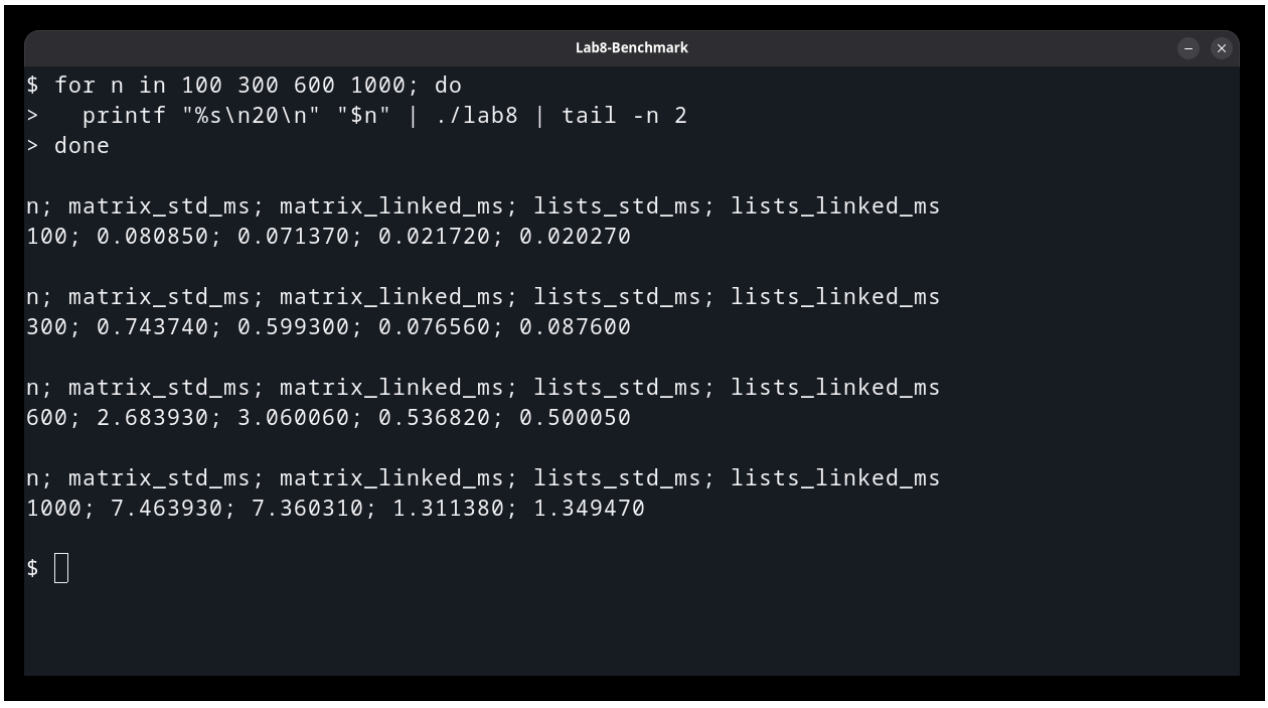


Рисунок 3 — Результаты измерения времени

Для матрицы время полного BFS составляет $O(n^2)$, для списков — $O(n + m)$, где m — число рёбер. Операции собственной очереди выполняются за O(1); стандартная очередь обеспечивает постоянное время добавления и удаления. Собственная очередь использует готовый массив узлов, поэтому отдельных выделений памяти при push нет. Дополнительная память обхода составляет O(n).

Различие замеров само по себе не доказывает устойчивого превосходства одного типа очереди.

При фиксированном p число рёбер растёт пропорционально n^2, поэтому на таких графах и списочная реализация в пределе имеет квадратичный рост. По эксперименту списки уменьшают стоимость просмотра отсутствующих рёбер.

Базовые проверки и вывод

Базовые проверки

Для четырёх изолированных вершин получен порядок 1 2 3 4; для одной вершины — 1; для полного графа из четырёх вершин — 1 2 3 4. Во всех случаях результаты четырёх режимов совпали. Проверены отказы при n = 0, n = 2001, нечисловом n, p = −1 и p = 101: программа завершилась с кодом 1 и сообщением о недопустимом числе.

Файлы и запуск

main.cpp содержит LinkedQueue, общий алгоритм traverse, измерение benchmark и точку входа. graph.h содержит структуры Node и Graph, выделение и освобождение памяти, генерацию, печать и проверку числового ввода. После запуска программа запрашивает n и p; параметры командной строки не используются.

```
g++ main.cpp -o lab8
./lab8
Введите число вершин (1..2000): 6
Введите вероятность ребра в процентах (0..100): 40
```

Вывод

Изучен обход графа в ширину. Реализованы четыре сочетания представления графа и очереди, включая связные списки смежности и собственную FIFO-очередь. Проверки подтвердили совпадение порядков обхода и посещение всех вершин, в том числе в несвязном графе. Измерено среднее время для графов разных порядков; выбор представления определяет объём просмотра соседей, а использование массива узлов исключает выделение памяти внутри операций собственной очереди.